

# RTFCLMGZN — System Upgrade Brief

Implementation plan for adding automated social distribution, direct image generation, and a desktop-to-cloud migration path to the existing AI news article pipeline.

Prepared for: Claude Code Desktop implementation session | July 2026

## 1. Purpose of This Document

RTFCLMGZN currently runs entirely on local hardware through Claude Code Desktop: a set of AI agents that research, write, and publish AI-news articles to the site on a schedule. This document specifies the next phase of work — adding two new agent roles for social distribution, switching image generation to a direct low-cost API, and removing dependency on paid "unified" social APIs in favor of direct, free platform APIs (Reddit + Meta).

This is a **build brief**, not a request to migrate to the cloud yet. All work described here should continue to run locally inside the existing Claude Code Desktop project. Section 6 lays out the future migration to cloud Routines as a separate, later phase — do not implement that section now.

## 2. What's Changing — Summary

| Area                  | Current State                                      | New State                                                                |
|-----------------------|----------------------------------------------------|--------------------------------------------------------------------------|
| Images                | Not yet implemented / Higgsfield considered        | Direct calls to Google's Gemini API ("Nano Banana," model gemini-2.5-... |
| Social distribution   | None — manual or not implemented                   | Two new agents: a Social Copy Generator and a Poster, added to the p...  |
| Social posting method | N/A                                                | Direct calls to Reddit API and Meta Graph API (Facebook + Instagram)     |
| Platforms covered     | N/A                                                | Reddit, Facebook, Instagram only. TikTok and YouTube explicitly exclu... |
| Hosting               | Local, always-on gaming PC via Claude Code Desktop | Stay local for now (Phase 1). Migration to Claude Code Routines (clou... |

### 3. New Agents To Add

Add these as two distinct new agents/steps in the existing pipeline, run immediately after an article is finalized. Keep them separate from each other and from the article-writing agent — each one is a distinct skill and keeping them isolated makes failures easier to diagnose (e.g. if posting fails, you don't want to have to regenerate the article and images to retry).

#### 3.1 — Image Agent (if not already present)

Runs immediately after the article text is finalized, before publication.

- Input: finished article text, plus any subject/scene description needed for the hero image and inline images.
- Calls the Gemini API (model: **gemini-2.5-flash-image**, or the newer gemini-3.1-flash-image "Nano Banana 2" family if available at implementation time — check current model IDs at [ai.google.dev/gemini-api/docs/image-generation](https://ai.google.dev/gemini-api/docs/image-generation) before hardcoding one) with a text prompt describing the scene, in the article's editorial visual style.
- If the article/brand has any recurring visual subject (a mascot, host avatar, consistent branding element), pass 1-2 saved reference images alongside the prompt in the same request for consistency — the same reference-image mechanism works whether the subject is a character, logo, or set style.
- Output: one or more image files, uploaded to wherever site media currently lives, with resulting URLs captured for the next step.
- Cost note: ~\$0.039/image on the standard model, no subscription — billed pay-as-you-go against a Google AI Studio API key with billing enabled. At typical volume (a handful of images per article) this should run a few dollars a month, not more.

#### 3.2 — Social Copy Generator Agent (new)

Runs after the article + images are finalized.

- Input: finished article text + image URL(s) + article metadata (title, URL, tags).
- Output: platform-specific copy, written as distinct passes rather than one generic blurb reused everywhere — each platform has different tone/length/format norms:

| Platform  | What to generate                                                                                                        |
|-----------|-------------------------------------------------------------------------------------------------------------------------|
| Reddit    | A title (no clickbait, matches how the target subreddit's community actually phrases post titles) + a body written like |
| Facebook  | A short link-preview-style post, 1-3 sentences, casual, may include 1-2 relevant hashtags.                              |
| Instagram | A caption that leads with a hook line, includes the core takeaway of the article, and ends with a small hashtag block   |

Output format: write this as a structured export file (JSON or Markdown with YAML frontmatter) into the project's content-export location, containing the article metadata, image URL(s), and the three platform-specific copy blocks, with a **status** field set to ready.

#### 3.3 — Poster Agent (new)

Runs after the Social Copy Generator produces a ready export file.

- Reads the export file(s) marked status: ready.

- Calls the Reddit API directly to submit the post to the identified subreddit(s).
- Calls the Meta Graph API directly to publish to the connected Facebook Page and linked Instagram Business account.
- On confirmed success from each platform's API response, updates the export file's status to posted (or logs a per-platform status if one platform succeeds and another fails — don't let one platform's failure block the other).
- Must include token-refresh handling for both platforms — do not assume a stored access token stays valid indefinitely; check current expiry conventions for each API at implementation time.
- Must be idempotent: re-running against an already-posted file should no-op, not double-post.

*Implementation note: sections 3.2 and 3.3 can be built as two separate agent steps in the same pipeline (recommended to start — simpler to build and debug one flow at a time), or merged into a single "social" agent later once the pipeline is stable. Do not build the posting logic before the copy-generation logic is producing good output — verify copy quality manually first.*

## 4. Social Platform Integration — Direct APIs, No Vendor

Do not integrate Ayrshare, Zernio, or any other unified social API vendor. At this project's scale (own accounts, two platforms, no multi-client management), these vendors add a recurring cost (\$19-599+/month depending on provider and tier) for problems this project doesn't have — they exist to solve pre-cleared platform audits for apps posting on behalf of many external users, and neither applies here.

### 4.1 — Reddit

- Free, no app review or audit process required for this use case.
- Register an app at [reddit.com/prefs/apps](https://reddit.com/prefs/apps) to obtain a client ID and secret.
- Use standard OAuth2 to obtain an access token; implement refresh handling.
- Verify current rate limits and submission requirements against Reddit's official API docs at implementation time, as these are subject to change.

### 4.2 — Meta Graph API (Facebook + Instagram, single integration)

- Free — no per-call charge from Meta.
- Requires: Instagram account converted to Business or Creator type, linked to a Facebook Page.
- Create an app at [developers.facebook.com](https://developers.facebook.com), add the Instagram Graph API product.
- **Important:** because this app will only ever post to accounts the owner directly manages, add the owner as Admin/Developer/Tester on the Meta app. This grants *Standard Access* for those roles' own accounts without going through Meta's full multi-week App Review process — that review is only required when an app posts on behalf of accounts it doesn't own/manage. Confirm this is still Meta's policy at implementation time, as platform review requirements are periodically revised.
- Publishing to Instagram is a two-step Graph API flow: create a media container, then publish it. Publishing to a Facebook Page is a single call. Both are documented under the current Graph API version at [developers.facebook.com](https://developers.facebook.com) — confirm the current version number before building, as Meta deprecates old versions on a rolling schedule.

### 4.3 — Explicitly Out of Scope

- TikTok — not needed per current project scope. (Note for future reference only: TikTok's Content Posting API requires passing an app audit before posts can be made public; unaudited apps are restricted to private-only posting.)
- YouTube — not needed per current project scope.

## 5. Updated Cost Summary

| Line item                             | Monthly cost | Notes                                                                        |
|---------------------------------------|--------------|------------------------------------------------------------------------------|
| Claude Pro plan (existing)            | \$20         | Covers interactive Claude Code Desktop use, and currently covers Routines/au |
| Image generation (Gemini/Nano Banana) | \$4-10       | Pay-as-you-go, no subscription. Scales with article + image volume.          |
| Reddit API                            | \$0          | Free, direct integration.                                                    |

| Line item                 | Monthly cost  | Notes                                                            |
|---------------------------|---------------|------------------------------------------------------------------|
| Meta Graph API            | \$0           | Free, direct integration, Standard Access via own-account roles. |
| Unified social API vendor | \$0           | Not used — removed from architecture.                            |
| TOTAL new recurring cost  | ~\$4-10/month | On top of the existing \$20/mo Claude Pro plan.                  |

## 6. Roadmap — Desktop Build Now, Cloud Migration Later

**Current phase: build and stabilize everything below on Claude Code Desktop, running locally, exactly as the project runs today.** Do not begin the cloud migration until Phase 1 is complete and stable. This roadmap exists so the eventual migration is a known, planned step rather than a surprise re-architecture.

### Phase 1 — Desktop build (current phase — build this now)

- Add the Image Agent (Section 3.1) to the existing pipeline; verify output quality manually for several articles before proceeding.
- Add the Social Copy Generator agent (Section 3.2); verify copy quality manually — read the generated Reddit/Facebook/Instagram copy for several articles before connecting live posting.
- Set up Reddit and Meta developer apps and obtain working API credentials (Section 4).
- Add the Poster agent (Section 3.3), initially in a dry-run/log-only mode that prints what it would post without actually calling the platform APIs, to verify formatting and target-selection logic.
- Switch the Poster agent to live mode once dry-run output looks correct across multiple articles.
- Run the full pipeline end-to-end on local hardware for at least 1-2 weeks, monitoring for failures, before considering it stable.

### Phase 2 — Cloud migration (later — do not start until Phase 1 is stable)

- Create a GitHub repository and push the project into it — this becomes the shared state store that both local Claude Code Desktop and cloud Routines can read from, since cloud Routines cannot access local disk at all.
- Re-verify current Claude Code Routines capabilities and constraints at [code.claude.com/docs](https://code.claude.com/docs) before migrating — the product may have changed since this brief was written. Key constraints to re-check: minimum schedule interval, whether local-file access has changed, and current billing treatment of automated usage.
- Migrate one agent at a time to a Routine, starting with the lowest-risk step (e.g. the article writer), testing each in the cloud before migrating the next.
- Store Reddit/Meta API credentials as environment variables in the Routine's cloud environment configuration rather than local files.
- Only after every agent runs successfully in the cloud for a trial period should the local always-on requirement be retired.
- Reassess at that time whether a self-hosted agent runtime (e.g. Hermes Agent, MIT-licensed, self-hosted) is worth adding for persistent cross-run memory/learning — this was evaluated during planning and deferred as unnecessary for current scale, since a single cloud Routine can call the Reddit/Meta APIs directly without a separate posting service.

*This document reflects platform capabilities, pricing, and policies as researched in July 2026. Before implementing Section 4 (platform APIs) and Section 6 (cloud migration), re-verify current terms directly against each platform's official developer documentation, as APIs, review requirements, and Anthropic's own product offerings change frequently.*